

Store Proxy and Integration Layer

Overview

The store backend layer is an Express module mounted into the main Yenkasa backend. It does not own the core commerce domain by itself; instead, it proxies many requests to another API base while also serving local store assets and a small amount of store-specific persistence.

Current implementation

- entry point: `store/yenkasa-store-server.js`
- mounted by: `server.js`
- proxy prefix: `/triciabales-api`
- upstream base: `TRICIABALES\_API\_BASE` or fallback `http://134.209.182.39:8080`
- local data:
 - `StoreProfile` model
 - local store-logo handling
 - local upload fallback for store media

Responsibilities

- expose store profile endpoints
- proxy buyer/seller/admin auth flows
- proxy product, order, refund, notification, payout, and paystack flows
- support store uploads and media updates
- provide role checks for super-admin-only store configuration updates

```
```mermaid
flowchart LR
 StorePages --> StoreProxy[yenkasa-store-server.js]
 StoreProxy --> StoreProfile[(Mongo StoreProfile)]
 StoreProxy --> Cloudinary
 StoreProxy --> Uploads[/uploads/store]
 StoreProxy --> Upstream[TRICIABALES_API_BASE]
```
```

Important routes

- `/triciabales-api/api/store-profile`
- `/triciabales-api/api/users/\*`
- `/triciabales-api/api/triciabales/\*`
- `/triciabales-api/api/orders/\*`
- `/triciabales-api/api/refunds/\*`
- `/triciabales-api/api/paystack/\*`
- `/triciabales-api/api/notifications/\*`
- `/triciabales-api/api/deliveries/uber/\*`

Known issues

- one large server module owns many unrelated commerce routes
- the proxy depends on another API base, which creates an additional operational dependency chain
- some defaults and fallback URLs are source-coded

Scaling concerns

- the store layer mixes proxying, authorization checks, upload processing, and local persistence
- failures can originate from the static store, the proxy layer, or the upstream commerce API
- route sprawl makes ownership harder as the store expands

Future improvements

1. split store proxy routes by domain: auth, catalog, orders, payouts, notifications
2. formalize upstream health and error handling
3. remove hardcoded fallback infrastructure values from source